PROJECT PROBLEM STATEMENT

FetalAI: Using Machine Learning To Predict And Monitor Fetal Health

Machine Learning • Flask Web Application • Healthcare Decision Support

Project Title	FetalAI: Using Machine Learning To Predict And Monitor Fetal Health
Document Type	Problem Statement
Domain	Healthcare • Prenatal Monitoring • Machine Learning
Technology Stack	Python, Scikit-learn, Flask, HTML/CSS/JavaScript
ML Model	Random Forest Classifier
Dataset Features	21 Numeric CTG Features
Output Classes	Normal • Suspect • Pathological
Document Status	Final • For Mentor Review

1. Background

During pregnancy, one of the main tools doctors use to check on a baby is the cardiotocogram, or CTG. It records the fetal heart rate and uterine contractions at the same time, which gives a picture of how the baby is doing. If the heart rate pattern looks unusual, it could mean the fetus is under stress and might need attention.

The problem is that reading a CTG trace is not straightforward. Different doctors sometimes reach different conclusions from the same trace. This inconsistency can lead to missed diagnoses or unnecessary interventions. Using machine learning to help classify CTG readings could make the process more consistent and give doctors a second opinion to consider.

This project is an attempt to build that kind of tool - not for real clinical use, but as a working demonstration that shows how a classification model can be trained on CTG data and then made accessible through a simple web interface.

2. Project Goal

The main goal is to build a complete, working system that takes CTG measurement values as input and outputs a predicted fetal health category. The three possible categories are:

NORMAL	The fetus appears healthy based on the CTG readings. No immediate concern.
SUSPECT	The readings are a bit unusual. A doctor should take a closer look before deciding anything.
PATHOLOGICAL	The readings suggest possible fetal distress. Urgent clinical attention would be needed.

The system should be accessible from a browser, easy to use, and clear about what the prediction means and what its limitations are.

3. Objectives

3.1 Machine Learning

- Look at the CTG dataset to understand what the features represent and how they relate to the health labels.
- Clean and prepare the data for training, including handling any missing values and splitting into train and test sets properly.
- Train a Random Forest model and compare it against a simpler baseline like a Decision Tree.
- Evaluate the model using accuracy, precision, recall, and F1-score. Pay special attention to how well it catches the Pathological cases.
- Save the trained model to a file so it can be loaded and used by the web app without retraining.

3.2 Web Application

- Build a Flask app with multiple pages: a landing/home page, a prediction form page, a results page, an about page, and a metrics/info page.
- Design the prediction form so all 21 input fields are labelled clearly and make sense to someone filling them in.
- Connect the form to the trained model so predictions happen in real time when the form is submitted.
- Show the result in a way that is easy to understand, including the predicted class and some indication of confidence.
- Include a page showing how the model performed during training so users know how reliable it is.

3.3 Documentation

- Write a README that explains how to set up the project and run it from scratch.
- Produce this problem statement and a final report covering the dataset, the approach taken, and what the results mean.

- Keep the code clean and commented so it is easy to follow during evaluation.

4. Problem Definition

At its core, this is a supervised multi-class classification problem. The dataset contains CTG recordings that were already labelled by expert obstetricians, so the task is to train a model to replicate that labelling on new, unseen inputs.

Input	21 numeric features from CTG recordings (baseline fetal heart rate, accelerations, decelerations, variability, histogram statistics, etc.).
Output	One of three labels: Normal, Suspect, or Pathological.
Dataset	2,126 records from the UCI Machine Learning Repository. The classes are imbalanced: roughly 78% Normal, 14% Suspect, and 8% Pathological.
Main Challenge	Because Pathological cases are rare in the data, the model might ignore them. Getting the model to correctly identify those cases is a priority.

5. Key Questions

- Which of the 21 CTG features are most useful for telling the three health categories apart?
- How well can a Random Forest model classify fetal health, and how does it compare to simpler models?
- How do we design an input form with 21 fields that does not feel overwhelming to use?
- What is the best way to show the prediction result so the user understands both what it means and its limitations?
- How should the app handle the fact that the model is not always right, especially for the minority Pathological class?

6. Target Users

This project is mainly aimed at three groups of people:

Medical Students	Students learning about prenatal monitoring who want to see how machine learning can be applied in a clinical context.
Healthcare Professionals	Professionals curious about what an AI decision-support demo looks like. This is not a clinical tool.

Course Instructors	The primary evaluators of this project. They will assess the code, the model, the web app, and the documentation.

7. Scope and Boundaries

7.1 In Scope

- ✓ Exploring and understanding the CTG dataset.
- ✓ Training a Random Forest classifier and testing it against a baseline model.
- ✓ Building a Flask web application with a working prediction interface.
- ✓ Saving the trained model so it can be used without retraining.
- ✓ Writing documentation explaining the project and how to run it.

7.2 Out of Scope

- ✗ Clinical validation or any kind of medical certification.
- ✗ Reading CTG signals directly from medical hardware.
- ✗ Deploying the app online or to any hospital system.
- ✗ Storing patient data or building any kind of user account system.

8. Deliverables

#	Deliverable	Description
1	**Trained Model File**	The saved Random Forest model (model.pkl) and scaler file, ready to load and run predictions.
2	**Training Script / Notebook**	Python code (or Jupyter notebook) that shows the full training process from loading data to saving the model.
3	**Flask Web Application**	A working local web app with all the pages described in the objectives section.
4	**Project Report**	A Word document explaining the dataset, what was done, the results, and any limitations.
5	**README File**	Step-by-step instructions for setting up and running the project on a new machine.
6	**Source Code**	All project files organised in a clear folder structure with a requirements.txt included.

9. Limitations and Responsible Use

Since this project deals with health data, it is important to be upfront about what the tool can and cannot do:

- This is a course project, not a medical device. It should not be used to make real decisions about patient care.
- The model is not perfect - it will sometimes get the prediction wrong, especially for Suspect and Pathological cases where the data is limited.
- Every results page in the app will include a clear disclaimer explaining that the output is for educational purposes only.
- The dataset used is publicly available and anonymised, so no real patient information is stored or processed by the app.
- The class imbalance in the dataset means the model has seen far fewer Pathological examples, which affects how well it detects them.

10. Success Criteria

The project will be considered complete and successful if:

- The model reaches at least 85% overall accuracy and correctly identifies at least 75% of Pathological cases in the test set.
- The Flask app runs without errors on a standard Python 3.8+ setup using the provided requirements file.
- All 21 CTG input fields work correctly, are clearly labelled, and the form returns a prediction without crashing.
- Someone else can follow the README and get the application running without needing to ask for help.
- The documentation is clear enough for a mentor to understand what was done, why, and what the results mean.

End of Problem Statement Document